ARM AArch64 Quick Reference

Arithmetic Instructions			
ADC{S}	rd, rn, rm	rd = rn + rm + C	
ADD{S}	rd, rn, op2	rd = rn + op2	S
ADR	Xd, $\pm rel_{21}$	Xd = PC + $rel^{\pm}$	
ADRP	Xd, $\pm rel_{33}$	Xd = $PC_{63:12}{:}0_{12}$ + $rel^{\pm}_{32:12}{:}0_{12}$	
CMN	rd, op2	rd + op2	S
CMP	rd, op2	rd − op2	S
MADD	rd, rn, rm, ra	rd = ra + rn × rm	
MNEG	rd, rn, rm	rd = − rn × rm	
MSUB	rd, rn, rm, ra	rd = ra − rn × rm	
MUL	rd, rn, rm	rd = rn × rm	
NEG{S}	rd, op2	rd = −op2	
NGC{S}	rd, rm	rd = −rm − ∼C	
SBC{S}	rd, rn, rm	rd = rn − rm − ∼C	
SDIV	rd, rn, rm	rd = rn $\bar{\div}$ rm	
SMADDL	Xd, Wn, Wm, Xa	Xd = Xa + Wn $\bar{\times}$ Wm	
SMNEGL	Xd, Wn, Wm	Xd = − Wn $\bar{\times}$ Wm	
SMSUBL	Xd, Wn, Wm, Xa	Xd = Xa − Wn $\bar{\times}$ Wm	
SMULH	Xd, Xn, Xm	Xd = $(Xn \bar{\times} Xm)_{127:64}$	
SMULL	Xd, Wn, Wm	Xd = Wn $\bar{\times}$ Wm	
SUB{S}	rd, rn, op2	rd = rn - op2	S
UDIV	rd, rn, rm	rd = rn ÷ rm	
UMADDL	Xd, Wn, Wm, Xa	Xd = Xa + Wn × Wm	
UMNEGL	Xd, Wn, Wm	Xd = − Wn × Wm	
UMSUBL	Xd, Wn, Wm, Xa	Xd = Xa − Wn × Wm	
UMULH	Xd, Xn, Xm	Xd = $(Xn \times Xm)_{127:64}$	
UMULL	Xd, Wn, Wm	Xd = Wn × Wm	

Bit Manipulation Instructions			
BFC	rd, #p, #n	$rd_{p+n-1:p} = 0_n$	2
BFI	rd, rn, #p, #n	$rd_{p+n-1:p} = rn_{n-1:0}$	
BFXIL	rd, rn, #p, #n	$rd_{n-1:0} = rn_{p+n-1:p}$	
CLS	rd, rn	rd = CountLeadingSigns(rn)	
CLZ	rd, rn	rd = CountLeadingZeros(rn)	
EXTR	rd, rn, rm, #p	$rd = rn_{p-1:0}{:}rm_{N-1:p}$	
RBIT	rd, rn	rd = ReverseBits(rn)	
REV	rd, rn	rd = BSwap(rn)	
REV16	rd, rn	for(n=0..1|3) rd_{Hn}=BSwap(rn_{Hn})	
REV32	Xd, Xn	Xd=BSwap($Xn_{63:32}$):BSwap($Xn_{31:0}$)	
REV64	Xd, Xn	Xd = BSwap(Xn)	O
{S,U}BFIZ	rd, rn, #p, #n	$rd = rn^{?}_{n-1:0} \ll p$	
{S,U}BFX	rd, rn, #p, #n	$rd = rn^{?}_{p+n-1:p}$	
{S,U}XT{B,H}	rd, Wn	$rd = Wn^{?}_{N0}$	
SXTW	Xd, Wn	Xd = $Wn^{\pm}$	

Logical and Move Instructions			
AND{S}	rd, rn, op2	rd = rn & op2	
ASR	rd, rn, rm	rd = rn $\bar{\gg}$ rm	
ASR	rd, rn, #i_6	rd = rn $\bar{\gg}$ i	
BIC{S}	rd, rn, op2	rd = rn & ∼op2	
EON	rd, rn, op2	rd = rn ⊕ ∼op2	
EOR	rd, rn, op2	rd = rn ⊕ op2	
LSL	rd, rn, rm	rd = rn ≪ rm	
LSL	rd, rn, #i_6	rd = rn ≪ i	
LSR	rd, rn, rm	rd = rn ≫ rm	
LSR	rd, rn, #i_6	rd = rn ≫ i	
MOV	rd, rn	rd = rn	S
MOV	rd, #i	rd = i	
MOVK	rd,#i_{16}{, sh}	$rd_{sh+15:sh}$ = i	
MOVN	rd,#i_{16}{, sh}	rd = ∼($i^{\emptyset} \ll$ sh)	
MOVZ	rd,#i_{16}{, sh}	rd = $i^{\emptyset} \ll$ sh	
MVN	rd, op2	rd = ∼op2	
ORN	rd, rn, op2	rd = rn | ∼op2	
ORR	rd, rn, op2	rd = rn | op2	
ROR	rd, rn, #i_6	rd = rn ⋙ i	
ROR	rd, rn, rm	rd = rn ⋙ rm	
TST	rn, op2	rn & op2	

Load and Store Instructions			
LDP	rt, rt2, [addr]	rt2:rt = $[addr]_{2N}$	
LDPSW	Xt, Xt2, [addr]	Xt = $[addr]^{\pm}_{32}$; Xt2 = $[addr+4]^{\pm}_{32}$	
LD{U}R	rt, [addr]	rt = $[addr]_N$	
LD{U}R{B,H}	Wt, [addr]	Wt = $[addr]^{\emptyset}_N$	
LD{U}RS{B,H}	rt, [addr]	rt = $[addr]^{\pm}_N$	
LD{U}RSW	Xt, [addr]	Xt = $[addr]^{\pm}_{32}$	
PRF{U}M	prfop, addr	Prefetch(addr, prfop)	
STP	rt, rt2, [addr]	$[addr]_{2N}$ = rt2:rt	
ST{U}R	rt, [addr]	$[addr]_N$ = rt	
ST{U}R{B,H}	Wt, [addr]	$[addr]_N$ = Wt_{N0}	

Addressing Modes (addr)		
xxP,LDPSW	[Xn{, #i_{7+s}}]	addr = Xn + $i^{\pm}_{6+s:s}{:}0_s$
xxP,LDPSW	[Xn], #i_{7+s}	addr=Xn; Xn+=$i^{\pm}_{6+s:s}{:}0_s$
xxP,LDPSW	[Xn, #i_{7+s}]!	Xn+=$i^{\pm}_{6+s:s}{:}0_s$; addr=Xn
xxR*,PRFM	[Xn{, #i_{12+s}}]	addr = Xn + $i^{\emptyset}_{11+s:s}{:}0_s$
xxR*	[Xn], #i_{9+s}	addr = Xn; Xn += $i^{\pm}_{8+s:s}{:}0_s$
xxR*	[Xn, #i_{9+s}]!	Xn += $i^{\pm}_{8+s:s}{:}0_s$; addr = Xn
xxR*,PRFM	[Xn,Xm{, LSL #0|s}]	addr = Xn + Xm ≪ s
xxR*,PRFM	[Xn,Wm,UXTW{ #0|s}]	addr = Xn + $Wm^{?}$ ≪ s
xxR*,PRFM	[Xn,rm,SXTX{ #0|s}]	addr = Xn + $rm^{\pm}$ ≪ s
xxUR*,PRFUM	[Xn{, #i_9}]	addr = Xn += $i^{\pm}$
LDR{SW},PRFM	$\pm rel_{21}$	addr = PC + $rel^{\pm}_{20:2}{:}0_2$

Branch Instructions		
B	rel_{28}	PC = PC + $rel^{\pm}_{27:2}{:}0_2$
Bcc	rel_{21}	if(cc) PC = PC + $rel^{\pm}_{20:2}{:}0_2$
BL	rel_{28}	X30 = PC + 4; PC += $rel^{\pm}_{27:2}{:}0_2$
BLR	Xn	X30 = PC + 4; PC = Xn
BR	Xn	PC = Xn
CBNZ	rn, rel_{21}	if(rn ≠ 0) PC += $rel^{\emptyset}_{20:2}{:}0_2$
CBZ	rn, rel_{21}	if(rn = 0) PC += $rel^{\emptyset}_{20:2}{:}0_2$
RET	{Xn}	PC = Xn
TBNZ	rn, #i, rel_{16}	if(rn_i ≠ 0) PC += $rel^{\pm}_{15:2}{:}0_2$
TBZ	rn, #i, rel_{16}	if(rn_i = 0) PC += $rel^{\pm}_{15:2}{:}0_2$

Conditional Instructions		
CCMN	rn, #i_5, #f_4, cc	if(cc) rn + i; else N:Z:C:V = f
CCMN	rn, rm, #f_4, cc	if(cc) rn + rm; else N:Z:C:V = f
CCMP	rn, #i_5, #f_4, cc	if(cc) rn − i; else N:Z:C:V = f
CCMP	rn, rm, #f_4, cc	if(cc) rn − rm; else N:Z:C:V = f
CINC	rd, rn, cc	if(cc) rd = rn + 1; else rd = rn
CINV	rd, rn, cc	if(cc) rd = ∼rn; else rd = rn
CNEG	rd, rn, cc	if(cc) rd = −rn; else rd = rn
CSEL	rd, rn, rm, cc	if(cc) rd = rn; else rd = rm
CSET	rd, cc	if(cc) rd = 1; else rd = 0
CSETM	rd, cc	if(cc) rd = ∼0; else rd = 0
CSINC	rd, rn, rm, cc	if(cc) rd = rn; else rd = rm + 1
CSINV	rd, rn, rm, cc	if(cc) rd = rn; else rd = ∼rm
CSNEG	rd, rn, rm, cc	if(cc) rd = rn; else rd = −rm

Operand 2 (op2)		
all	*rm*	rm
all	rm, LSL #i_6	rm ≪ i
all	rm, LSR #i_6	rm ≫ i
all	rm, ASR #i_6	rm $\bar{\gg}$ i
logical	rm, ROR #i_6	rm ⋙ i
arithmetic	Wm, {S,U}XTB{ #i_3}	$Wm^{?}_{B0} \ll$ i
arithmetic	Wm, {S,U}XTH{ #i_3}	$Wm^{?}_{H0} \ll$ i
arithmetic	Wm, {S,U}XTW{ #i_3}	$Wm^{?} \ll$ i
arithmetic	Xm, {S,U}XTX{ #i_3}	$Xm^{?} \ll$ i
arithmetic	#i_{12}	$i^{\emptyset}$
arithmetic	#i_{24}	$i^{\emptyset}_{23:12}{:}0_{12}$
AND,EOR,ORR,TST	#mask	mask

Notes for Instruction Set	
O	Optional feature
S	SP/WSP used as operand(s) instead of XZR/WZR
1,2,3,4,5,6	Introduced in ARMv8.{1..6}

Atomic Instructions (ARMv8.1)		
CAS{A}{L}	rs, rt, [Xn]	if (rs = $[Xn]_N$) $[Xn]_N$ = rt
CAS{A}{L}{B,H}	Ws, Wt, [Xn]	if (Ws_{N0} = $[Xn]_N$) $[Xn]_N$ = Wt_{N0}
CAS{A}{L}P	rs,rs2,rt,rt2,[Xn]	if (rs2:rs = $[Xn]_{2N}$) $[Xn]_{2N}$ = rt2:rt
LDao{A}{L}	rs, rt, [Xn]	rt = $[Xn]_N$; $[Xn]_N$ = ao($[Xn]_N$, rs)
LDao{A}{L}{B,H}	Ws, Wt, [Xn]	Wt=$[Xn]^{\emptyset}_N$; $[Xn]_N$=ao($[Xn]_N$,Ws_{N0})
STao{A}{L}	rs, [Xn]	$[Xn]_N$ = ao($[Xn]_N$, rs)
STao{A}{L}{B,H}	Ws, [Xn]	$[Xn]_N$ = ao($[Xn]_N$, Ws_{N0})
SWP{A}{L}	rs, rt, [Xn]	rt = $[Xn]_N$; $[Xn]_N$ = rs
SWP{A}{L}{B,H}	Ws, Wt, [Xn]	Wt = $[Xn]^{\emptyset}_N$; $[Xn]_N$ = Ws_{N0}

Atomic Operations (ao)	
ADD [Xn] + rs	SMAX [Xn] $\tilde{>}$ rs ? [Xn] : rs
CLR [Xn] & ∼rs	SMIN [Xn] $\tilde{<}$ rs ? [Xn] : rs
EOR [Xn] ⊕ rs	UMAX [Xn] > rs ? [Xn] : rs
SET [Xn] | rs	UMIN [Xn] < rs ? [Xn] : rs

Checksum Instructions		
CRC32{B,H,W}	Wd, Wn, Wm	Wd=CRC32(Wn,0x04c11db7,Wm_{N0}) O
CRC32X	Wd, Wn, Xm	Wd = CRC32(Wn,0x04c11db7,Xm) O
CRC32C{B,H,W}	Wd, Wn, Wm	Wd=CRC32(Wn,0x1edc6f41,Wm_{N0}) O
CRC32CX	Wd, Wn, Xm	Wd = CRC32(Wn,0x1edc6f41,Xm) O

Hint Instructions		
BTI {j}{c}	SetBTypeNext({j}{c})	5
DGH	DataGatheringHint()	O
NOP		
SEV	SendEvent()	
SEVL	EventRegisterSet()	
WFE	WaitForEvent()	
WFI	WaitForInterrupt()	
YIELD		

Flag Manipulation Instructions		
CFINV	C = ∼C	4
RMIF Xn, #i_6, #m_4	for(n=0..3) if(m_n) NZCV$_n$ = Xn_{i+n}	4
SETF{8,16} Wn	N = Wn_{N-1}; Z = Wn=0 ? 1 : 0; V = Wn_{N-1} ⊕ Wn_{N-2}	4

DMB and DSB Options (barrierop)	
OSH{,LD,ST}	Outer shareable, {all,load,store}
NSH{,LD,ST}	Non-shareable, {all,load,store}
ISH{,LD,ST}	Inner shareable, {all,load,store}
LD	Full system, load
ST	Full system, store
SY	Full system, all

Load and Store Instructions with Attribute			
LD{A}XP	rt, rt2, [Xn]	rt2:rt = [Xn, <SetExclMonitor>]$_{2N}$	
LD{A}{X}R	rt, [Xn]	rt = [Xn, <SetExclMonitor>]$_N$	
LD{A}{X}R{B,H}	Wt, [Xn]	Wt = [Xn, <SetExclMonitor>]$^{\emptyset}_N$	
LDLAR{,B,H}	rt, [Xn]	rt = [Xn, <LimitedOrdered>]$^{\emptyset}_N$	1
LDARP{,B,H}	rt, [Xn]	rt = [Xn, <Ordered>]$^{\emptyset}_N$	3
LDAPUR{,B,H}	rt, [addr]	rt = [addr, <Ordered>]$^{\emptyset}_N$	4
LDAPURS{B,H,W}	rt, [addr]	rt = [addr, <Ordered>]$^{\pm}_N$	4
LDNP	rt, rt2, [addr]	rt2:rt = [addr, <Temp>]$_{2N}$	
LDTR	rt, [addr]	rt = [addr, <Unpriv>]$_N$	
LDTR{B,H}	Wt, [addr]	Wt = [addr, <Unpriv>]$^{\emptyset}_N$	
LDTRS{B,H}	rt, [addr]	rt = [addr, <Unpriv>]$^{\pm}_N$	
LDTRSW	Xt, [addr]	Xt = [addr, <Unpriv>]$^{\pm}_{32}$	
STLLR{,B,H}	rt, [Xn]	[Xn, <LimitedOrdered]$^{\emptyset}_N$ = rt	1
STLR	rt, [Xn]	[Xn, <Release>]$_N$ = rt	
STLR{B,H}	Wt, [Xn]	[Xn, <Release>]$_N$ = Wt_{N0}	
ST{L}XP	Wd, rt, rt2, [Xn]	[Xn,<Excl>]$_{2N}$=rt:rt2;Wd=fail?1:0	
ST{L}XR	Wd, rt, [Xn]	[Xn, <Excl>]$_N$=rt; Wd=fail?1:0	
ST{L}XR{B,H}	Wd, Wt, [Xn]	[Xn,<Excl>]$_N$=Wt_{N0}; Wd=fail?1:0	
STNP	rt, rt2, [addr]	[addr, <Temp>]$_{2N}$ = rt2:rt	
STTR	rt, [addr]	[addr, <Unpriv>]$_N$ = rt	
STTR{B,H}	Wt, [addr]	[addr, <Unpriv>]$_N$ = Wt_{N0}	

Addressing Modes (addr) with Attribute		
xxNP	[Xn{, #i_{7+s}}]	addr = Xn + $i^{\pm}_{6+s:s}$:0_s
xxTR*,LDAPUR*	[Xn{, #i_9}]	addr = Xn += $i^{\pm}$
LDRA	[Xn{, #i_{12}}]{!}	addr = Xn + $i^{\pm}_{11:3}0_3$; if(!) Xn=addr
LDRA	[Xn], #i_{12}	addr = Xn; Xn += $i^{\pm}_{11:3}$:0_3
STGP	[Xn{, #i_{11}}]{!}	addr = Xn + $i^{\pm}_{10:4}$:0_4; if(!) Xn=addr
STGP	[Xn], #i_{11}	addr = Xn; Xn += $i^{\pm}_{10:4}$:0_4
LDG	[Xn{, #i_{13}}]	addr = Xn + $i^{\pm}_{12:4}$:0_4
ST{Z}{2}G	[Xn{, #i_{13}}]{!}	addr = Xn + $i^{\pm}_{12:4}$:0_4; if(!) Xn=addr
ST{Z}{2}G	[Xn], #i_{13}	addr = Xn; Xn += $i^{\pm}_{12:4}$:0_4

Barrier Instructions			
CLREX	{#i_4}	ClearExclusiveLocal()	
CSDB		ConsumeOfSpeculativeDataBarrier()	
DMB	barrierop	DataMemoryBarrier(barrierop)	
DSB	barrierop	DataSyncBarrier(barrierop)	
ESB		ErrorSyncBarrier()	2
ISB	{SY}	InstructionSyncBarrier(SY)	
PSB	CSYNC	ProfilingSynchronizationBarrier()	2
PSSBB		SpeculativeStoreBypassBarrierToPA()	
SB		SpeculationBarrier()	O
SSBB		SpeculativeStoreBypassBarrierToVA()	
TSB	CSYNC	TraceSynchronizationBarrier()	4

Memory Tagging Instructions (ARMv8.5)			
ADDG	Xd,Xn,#i_{10},#j_4	Xd = Xn + $i^{\emptyset}_{9:4}$:0_4; $Xd_{59:56}$ = ChooseTag($Xn_{59:56}$, j)	S
CMPP	Xn, Xm	$Xn^{\pm}_{55:0}$ − $Xm^{\pm}_{55:0}$	S,O
GMI	Xd, Xn, Xm	Xd = $Xm_{63:60}$:$Xn_{59:56}$:$Xm_{55:0}$	S,O
IRG	Xd, Xn{, Xm}	Xd = $Xn_{63:60}$:RndTag(Xm):$Xn_{55:0}$	S
LDG	Xt, [addr]	$Xt_{59:56}$ = GetTag(addr)	
LDGM	Xt, [Xn]	Xt=0;for(n=0..((1≪(BS−2))−1) $Xt_{4n+3:4n}$=GetTag(Xn+16n)	O
SUBG	Xd,Xn,#i_{10},#j_4	Xd = Xn − $i^{\emptyset}_{9:4}$:0_4; $Xd_{59:56}$ = ChooseTag($Xn_{59:56}$, j)	S
SUBP{S}	Xd, Xn, Xm	Xd = $Xn^{\pm}_{55:0}$ - $Xm^{\pm}_{55:0}$	S,O
STG	Xt, [addr]	SetTag(addr, $Xt_{59:56}$)	
STZG	Xt, [addr]	$[addr]_{128}$=0; SetTag(addr,$Xt_{59:56}$)	
ST2G	Xt, [addr]	SetTag(addr, $Xt_{59:56}$); SetTag(addr+16, $Xt_{59:56}$)	
STZ2G	Xt, [addr]	$[addr]_{256}$=0;SetTag(addr,$Xt_{59:56}$); SetTag(addr+16, $Xt_{59:56}$)	
STG{Z}M	Xt, [Xn]	for(n = 0..((1≪(BS−2))−1) SetTag(Xn+16n, $Xt_{4n+3:4n}$)	O
STGP	Xt, Xt2, [addr]	$[addr]_{128}$ = Xt2:Xt; SetTag(addr, $addr_{59:56}$)	

Pointer Authentication Instructions (ARMv8.3)			
AUT{D,I}k	Xd, Xn	Xd = Auth(Xd, Xn, k, {D,I})	S
AUT{D,I}	Zk Xd	Xd = Auth(Xd, 0, k, {D,I})	
AUTIk1716		X17 = Auth(X17, X16, k, I)	
AUTIkSP		X30 = Auth(X30, SP, k, I)	
AUTIkZ		X30 = Auth(X30, 0, k, I)	
BRAk	Xn, Xm	PC = Auth(Xn, Xm, k, I)	S
BRAkZ	Xn	PC = Auth(Xn, 0, k, I)	
BLRAk	Xn, Xm	X30=PC+4; PC = Auth(Xn,Xm,k,I)	S
BLRAkZ	Xn	X30=PC+4; PC = Auth(Xn,0,k,I)	
ERETAk		PC=Auth(ELR_ELn, SP, k, I); PSTATE = SPSR_ELn	
LDRAk	Xt, [addr]	Xt = [Auth(addr, 0, k, D)]	
PAC{D,I}k	Xd, Xn	Xd = AddPAC(Xd, Xn ,k, {D,I})	S
PAC{D,I}Zk	Xd	Xd = AddPAC(Xd, 0, k, {D,I})	
PACGA	Xd, Xn, Xm	Xd = CompPAC(Xn,Xm,APGAKey)	S
PACIkSP		X30 = AddPAC(X30, SP, k, I)	
PACIkZ		X30 = AddPAC(X30, 0, k, I)	
PACIk1716		X17 = AddPAC(X17, X16, k, I)	
RETAk		PC = Auth(X30, SP, k, I)	
XPAC{D,I}	Xd	Xd = StripPAC(Xd, {D,I})	
XPACLRI		X30 = StripPAC(X30, I)	

System Instructions			
AT S1{2}E{0..3}{R,W}{P}, Xn		PAR_EL1 = AddrTrans(Xn)	
BRK	$\#i_{16}$	SoftwareBreakpoint(i)	
CFP	RCTX, Xn	Restrict(ControlFlow, Xn)	O
CPP	RCTX, Xn	Restrict(CachePrefetch, Xn)	O
DVP	RCTX, Xn	Restrict(DataValue, Xn)	O
ERET		PC=ELR_ELn;PSTATE=SPSR_ELn	
HVC	$\#_{16}$	CallHypervisor(i)	
MRS	Xn, sysreg	Xn = sysreg	
MSR	sysreg, Xn	sysreg = Xn	
MSR	SPSel, $\#i_1$	PSTATE.SP = i	
MSR	DAIFSet, $\#i_4$	PSTATE.DAIF |= i	
MSR	DAIFClr, $\#i_4$	PSTATE.DAIF &= ∼i	
SMC	$\#i_{16}$	CallSecureMonitor(i)	
SVC	$\#i_{16}$	CallSupervisor(i)	

Cache and TLB Maintenance Instructions			
DC	{C,CI,I}{,G,GD}SW, Xx	DC/Tag clean/inv by Set/Way	
DC	{C,CI,I}{,G,GD}VAC, Xx	DC/Tag clean/inv by VA to PoC	
DC	C{,G,GD}VA{D}P, Xx	DC/Tag clean by VA to Po{D}P	2
DC	CVAU, Xx	DC clean by VA to PoU	
DC	{G,GZ,Z}VA, Xx	DC zero/Tag by VA (DCZID_EL0)	
IC	IALLU{IS}	IC inv all to PoU	
IC	IVAU, Xx	IC inv VA to PoU	
TLBI	tblop{,IS,OS}, Xx	TLB invalidate {to inner/outer share}	

TLBI Options	
ALLE{1..3}	All
ASIDE1	by ASID
VMALL{S12}E1	By VMID, all, stage 1 {& 2}
{R}IPAS2{L}E1	Range by IPA {, last level only}
{R}VA{L}E{1..3}	{Range} by VA{, last level only}
{R}VAA{L}E1	{Range} by VA, all ASID{, last level only}

Registers		
X0-X7	W0-W7	Arguments and return values
X8	W8	Indirect result
X9-X15	W9-W15	Temporary
X16-X17	W16-W17	Intra-procedure-call temporary
X18	W18	Platform defined use
X19-X28	W19-W28	Temporary (must be preserved)
X29	W29	Frame pointer (must be preserved)
X30	W30	Return address
SP	WSP	Stack pointer
XZR	WZR	Zero
PC		Program counter

Keys	
N	Operand bit size (8, 16, 32 or 64)
s	Operand log byte size (0=byte,1=hword,2=word,3=dword)
rd, rn, rm, rt	General register of either size (Wn or Xn)
prfop	P{LD,LI,ST}L{1..3}{KEEP,STRM}
k	Pointer Authentication Key (A or B)
{,sh}	Optional halfword left shift (LSL #{16,32,48})
$val^{\pm}$, $val^{\emptyset}$, $val^{?}$	Value is sign/zero extended (? depends on instruction)
$\bar{\times}$ $\bar{\div}$ $\bar{\gg}$ $\bar{>}$ $\bar{<}$	Operation is signed

Condition Codes (cc)		
EQ	Equal	Z
NE	Not equal	!Z
CS/HS	Carry set, Unsigned higher or same	C
CC/LO	Carry clear, Unsigned lower	!C
MI	Minus, Negative	N
PL	Plus, Positive or zero	!N
VS	Overflow	V
VC	No overflow	!V
HI	Unsigned higher	C & !Z
LS	Unsigned lower or same	!C | Z
GE	Signed greater than or equal	N = V
LT	Signed less than	N ≠ V
GT	Signed greater than	!Z & N = V
LE	Signed less than or equal	Z | N ≠ V
AL	Always	1

Process State (PSTATE) and Saved Program Status (SPSR)			
SP	0x00000001	Stack pointer register selection bit	
EL	0x0000000C	Current Exception level	
nRW	0x00000010	Execution state (0=AArch64, 1=AArch32)	
F	0x00000040	FIQ interrupt mask	
I	0x00000080	IRQ interrupt mask	
A	0x00000100	SError interrupt mask	
D	0x00000200	Debug exception mask	
BTYPE	0x00000C00	Branch target identification bit	5
SSBS	0x00001000	Speculative Store Bypass Safe bit	O
IL	0x00100000	Illegal Execution	
SS	0x00200000	Software Step bit	
PAN	0x00400000	Privileged Access Never state bit	1
UAO	0x00800000	User Access Override bit	2
DIT	0x01000000	Data Independent Timing bit	4
TCO	0x02000000	Tag Check Override bit	O,5
V	0x10000000	Overflow condition flag	
C	0x20000000	Carry condition flag	
Z	0x40000000	Zero condition flag	
N	0x80000000	Negative condition flag	

Special Purpose Registers		
SPSR_EL{1..3}	Process state on exception entry to EL{1..3}	
ELR_EL{1..3}	Exception return address from EL{1..3}	
SP_EL{0..2}	Stack pointer for EL{0..2}	
SPSel	SP selection (0: SP=SP_EL0, 1: SP=SP_ELn)	
CurrentEL	Current Exception level (at bits 3..2)	RO
DAIF	Current interrupt mask bits (at bits 9..6)	
SSBS	Speculative Store Bypass Safe (at bit 12)	O
PAN	Privileged Access Never (at bit 22)	1
UAO	User Access Override (at bit 23)	2
DIT	Data Independent Timing (at bit 24)	4
TCO	Tag Check Override (at bit 25)	5
NZCV	Condition flags (at bits 31..28)	
FPCR	Floating-point operation control	
FPSR	Floating-point status	

Exception Vectors	
0x000,0x080,0x100,0x180	{Sync,IRQ,FIQ,SError} from cur lvl with SP_EL0
0x200,0x280,0x300,0x380	{Sync,IRQ,FIQ,SError} from cur lvl with SP_ELn
0x400,0x480,0x500,0x580	{Sync,IRQ,FIQ,SError} from lower lvl using A64
0x600,0x680,0x700,0x780	{Sync,IRQ,FIQ,SError} from lower lvl using A32

Exception Classes	
0x00	Unknown reason
0x01	Trapped WFI or WFE instruction execution
0x07	Trapped access to SIMD/FP
0x09	Trapped access to PAuth instruction
0x0e	Illegal Execution state
0x11,0x15	SVC instruction execution in AArch{32,64} state
0x12,0x16	HVC instruction execution in AArch{32,64} state
0x13,0x17	SMC instruction execution in AArch{32,64} state
0x18	Trapped MSR, MRS, or System instruction execution
0x19	Trapped access to SVE functionallity
0x1a	Trapped ERET or ERETAk execution
0x1c	Pointer authentication failure
0x1f	Implementation defined exception to EL3
0x20,0x21	Instruction Abort from {lower,current} level
0x22,0x26	{PC,SP} alignment fault
0x24,0x25	Data Abort from {lower,current} level
0x28,0x2c	Trapped float-point exception from AArch{32,64} state
0x2f	SError interrupt
0x30,0x31	Breakpoint exception from {lower,current} level
0x32,0x33	Software Step exception from {lower,current} level
0x34,0x35	Watchpoint exception from {lower,current} level
0x38,0x3c	{BKPT,BRK} instruction excecution from AArch{32,64} state

AArch64 Floating-point and Advanced SIMD Extensions

SIMD Load/Store Instructions		
LD1	$\{vt_{BHSD}\}$, [addr]	$Vt = [addr]_{nz}$
LD1	$\{vt_{BHSD}$, ...$\}$, [addr]	$\{\{Vt''':\}Vt'':\}Vt':Vt = [addr]_{\{2;3;4\}nz}$
LD1	$\{Vt.t_{BHSD}[i]\}$, [addr]	$Vt_{ti} = [addr]_z$
LD1R	$\{Vt_{BHSD}\}$, [addr]	$Vt_{t*} = [addr]_z$
LD{2,3,4}	$\{vt_{BHSD}$, ...$\}$, [addr]	$\{\{Vt'''_{te}:Vt''_{te}:...Vt'_{t0}:Vt_{t0} = [addr]_{nz}$
LD{2,3,4}	$\{Vt.t_{BHSD}[i]$,...$\}$,[addr]	$\{\{Vt'''_{ti}:\}Vt''_{ti}:\}Vt'_{ti}:Vt_{ti} = [addr]_{nz}$
LD{2,3,4}R	$\{vt_{BHSD}$, ...$\}$, [addr]	$\{\{Vt'''_{t*}:\}Vt''_{t*}:\}Vt'_{t*}:Vt_{t*}=[addr]_{nz}$
LD{N}P	qt_{SDQ}, $qt2_d$, [addr]	$Vt2_{t0}:Vt_{t0} = [addr]_{2z}$
LD{U}R	qt_{BHSDQ}, [addr]	$Vt_{t0} = [addr]_z$
ST1	$\{vt_{BHSD}\}$, [addr]	$[addr]_{nz} = Vt$
ST1	$\{vt_{BHSD}$, ...$\}$, [addr]	$[addr]_{\{2;3;4\}nz}=\{\{Vt''':\}Vt'':\}Vt':Vt$
ST1	$\{Vt.t_{BHSD}[i]\}$, [addr]	$[addr]_z = Vt_{ti}$
ST{2,3,4}	$\{vt_{BHSD}$, ...$\}$, [addr]	$[addr]_{nz} = \{\{Vt'''_{te}:Vt''_{te}:...Vt'_{t0}:Vt_{t0}$
ST{2,3,4}	$\{Vt.t_{BHSD}[i]$,...$\}$,[addr]	$[addr]_{nz} = \{\{Vt'''_{ti}:\}Vt''_{ti}:\}Vt'_{ti}:Vt_{ti}$
ST{N}P	qt_{SDQ}, $qt2_d$, [addr]	$[addr]_{2z} = Vt2_{t0}:Vt_{t0}$
ST{U}R	qt_{BHSDQ}, [addr]	$[addr]_z = Vt_{t0}$

SIMD Load/Store Addressing Modes		
xx{1,2,3,4}{R}	[Xn]	addr = Xn
xx{1,2,3,4}{R}	[Xn], #i	addr = Xn; Xn += nz ÷ 8
xx{1,2,3,4}{R}	[Xn], Xm	addr = Xn; Xn += Xm
xxNP	[Xn{, #i_{7+s}}]	addr = Xn + $i^{\pm}_{6+s:s}:0_s$
xxP	[Xn, #i_{7+s}]!	Xn =+ $i^{\pm}_{6+s:s}:0_s$; addr = Xn
xxP	[Xn], #i_{7+s}	addr = Xn; Xn =+ $i^{\pm}_{6+s:s}:0_s$
xxR	[Xn], #i_9	addr = Xn; Xn =+ $i^{\pm}$
xxR	[Xn, #i_9]!	Xn =+ $i^{\pm}$; addr = Xn
xxR	[Xn{, #i_{12+s}}]	addr = Xn + $i^{\emptyset}_{11+s:s}:0_s$
xxR	[Xn, Xm{, LSL #{0,s}}]	addr = Xn + Xm ≪ {0,s}
xxR	[Xn, Wm{, sXTW{ #0,s}}]	addr = Xn + rm^s ≪ {0,s}
xxR	[Xn, Xm{, SXTX{ #0,s}}]	addr = Xn + $rm^{\pm}$ ≪ {0,s}
xxUR	[Xn{, #i_9}]	addr = Xn + $i^{\pm}$
LDR	±rel_{21}	addr = PC + $rel^{\pm}_{20:2}:0_2$

SIMD Misc Instructions			
PMUL	vd_B, vn_d, vm_d	$Vd_{B*} = Vn_{B*} * Vm_{B*}$	
PMULL{2}	vd_{HQ}, vn_{BD}, vm_n	$Vd_{t*} = Vn_{t'*} * Vm_{t'*}$	
sDOT	vd_S, vn_B, vm_n	$Vd_{S*}=\sum_{k=0}^{3}(Vn_{B4*+k}\times^{s}Vm_{B4*+k})$	2,I,O
sMMLA	Vd.4S,Vn.16B,Vm.16B	$Vd_{S[2;2]} += Vn_{B[2;8]} \times^{s} Vm_{B[8;2]}$	2,O
SUDOT	vd_S, vn_B, Vm.4B[i]	$Vd_{S*}=\sum_{k=0}^{3}(Vn^{\pm}_{B4*+k}\bar{\times}Vm^{\emptyset}_{Bi+k})$	2,O
URECPE	vd_S, vn_d	Vd_{S*} = RecipEstimate(Vn_{S*})	
URSQRTE	vd_S, vn_d	Vd_{S*} = RecipSqrtEstimate(Vn_{S*})	
USDOT	vd_S, vn_B, vm_n	$Vd_{S*}=\sum_{k=0}^{3}(Vn^{\emptyset}_{B4*+k}\bar{\times}Vm^{\pm}_{B4*+k})$	2,I,O
USMMLA	Vd.4S,Vn.16B,Vm.16B	$Vd_{S[2;2]} += Vn^{\emptyset}_{B[2;8]} \times Vm^{\pm}_{B[8;2]}$	2,O

Floating Point Arithmetic Instructions			
FABD	vd_{HSD}, vn_d, vm_d	$Vd_{t*} = \|Vn_{t*} - Vm_{t*}\|$	M
FABS	vd_{HSD}, vn_d	$Vd_{t*} = \|Vn_{t*}\|$	M
FACGE	vd_{HSD}, vn_d, vm_d	$Vd_{t*}=(\|Vn_{t*}\|\geq\|Vm_{t*}\|)$? 1_z : 0_z	M
FACGT	vd_{HSD}, vn_d, vm_d	$Vd_{t*}=(\|Vn_{t*}\|>\|Vm_{t*}\|)$? 1_z : 0_z	M
FADD	vd_{HSD}, vn_d, vm_d	$Vd_{t*} = Vn_{t*} + Vm_{t*}$	M
FADDP	vd_{HSD}, vn_d, vm_d	$Vd_{t*}=(Vm:Vn)_{t2*}+(Vm:Vn)_{t2*+1}$	M
FDIV	vd_{HSD}, vn_d, vm_d	$Vd_{t*} = Vn_{t*} \div Vm_{t*}$	M
FMLa	vd_{HSD}, vn_d, vm_d	$Vd_{t*} \pm= Vn_{t*} \times Vm_{t*}$	I,M
FMLaL{2}	vd_S, vn_H, vm_H	$Vd_{S*} \pm= Vn_{H*} \times Vm_{H*}$	2,I,O
FM{N}ADD	qd_{HSD},qn_d,qm_d,qa_d	$Vd_{t0} = \pm Va_{t0} + Vn_{t0} \times Vm_{t0}$	
FM{N}SUB	qd_{HSD},qn_d,qm_d,qa_d	$Vd_{t0} = \pm Va_{t0} - Vn_{t0} \times Vm_{t0}$	
FNEG	vd_{HSD}, vn_d	$Vd_{t*} = -Vn_{t*}$	M
F{N}MUL{X}	qd_{HSD}, qn_d, qm_d	$Vd_{t0} = \pm Vn_{t0} \times Vm_{t0}$	
FSQRT	vd_{HSD}, vn_d	Vd_{t*} = Sqrt(Vn_{t*})	M
FSUB	vd_{HSD}, vn_d, vm_d	$Vd_{t*} = Vn_{t*} - Vm_{t*}$	M

Floating Point Move Instructions			
FCSEL	qd_{HSD}, qn_d, qm_d, cc	Vd_{t0} = cc ? Vn_{t0} : Vm_{t0}	
FMOV	qd_{HSD}, qn_d	$Vd_{t0} = Vn_{t0}$	
FMOV	rd, qn_{HSD}	rd = $Vn^{\emptyset}_{t0}$	
FMOV	Xd, Dn[1]	Xd = Vn_{D1}	
FMOV	qd_{HSD}, rn_d	$Vd_{t0} = rn_{t0}$	
FMOV	Dd[1], Xn	$Vd_{D1} = Xn$	
FMOV	vd_{HS}, #i	$Vd_{t*} = i$	M_{HSD}

Floating-point Control Registers (FPCR)			
IOE	0x00000100	Invalid operation exception trap enable	
DZE	0x00000200	Divide by zero exception trap enable	
OFE	0x00000400	Overflow exception trap enable	
UFE	0x00000800	Underflow exception trap enable	
IXE	0x00001000	Inexact exception trap enable	
IDE	0x00008000	Input denormal exception trap enable	
FZ16	0x00080000	Flush-to-zero mode on half-precision	2
RMode	0x00c00000	Rounding mode (nearest, plus inf, minus inf, zero)	
FZ	0x01000000	Flush-to-zero mode	
DN	0x02000000	Default NaN mode	
AHP	0x04000000	Alternate half-precision format	2

Floating-point Status Registers (FPSR)		
IOC	0x00000001	Invalid operation cumulative exception
DZC	0x00000002	Divide by zero cumulative exception
OFC	0x00000004	Overflow cumulative exception
UFC	0x00000008	Underflow cumulative exception
IXC	0x00000010	Inexact cumulative exception
IDC	0x00000080	Input denormal cumulative exception
QC	0x08000000	Cumulative saturation

Floating Point Conversion Instructions			
BFCVT{2}	vd_H, vn_S	Vd_{H*} = Float2BFloat16(Vn_{S*})	6,M
FCVT	qd_{HSD}, qn_{HSD}	Vd_{t0} = Float2Float($Vn_{t'0}$)	
FCVTrs	vd_{HSD}, vn_d	Vd_{t*} = Float2Int(Vn_{t*}, s)	M,R_d
FCVTL{2}	vd_{SD}, vn_{HS}	Vd_{t*} = Float2Float($Vn_{t'*}$)	
FCVTN{2}	vd_{HS}, vn_{SD}	Vd_{t*} = Float2Float($Vn_{t'*}$)	
FCVTXN{2}	vd_S, vn_D	Vd_{S*} = Float2Float(Vn_{D*})	M
FCVTZs	vd_{HSD}, vn_d	Vd_{t*} = Float2Int(Vn_{t*}, s)	M,R_d
FCVTZs	vd_{HSD}, vn_d, #i	Vd_{t*} = Float2Fixed(Vn_{t*}, i, s)	M,R_d
FJCVTZS	Wd, Dn	Wd = Float2FixedJS(Vn_{D0})	3
FRINT32X	vd_{SD}, vn_d	Vd_{t*} = Round2Int32(Vn_{t*},FPCR)	5,M
FRINT32Z	vd_{SD}, vn_d	Vd_{t*} = Round2Int32(Vn_{t*}, Z)	5,M
FRINT64X	vd_{SD}, vn_d	Vd_{t*} = Round2Int64(Vn_{t*},FPCR)	5,M
FRINT64Z	vd_{SD}, vn_d	Vd_{t*} = Round2Int64(Vn_{t*}, Z)	5,M
FRINTr	vd_{HSD}, vn_d	Vd_{t*} = Round2Int(Vn_{t*}, r)	M
FRINTI	vd_{HSD}, vn_d	Vd_{t*} = Round2Int(Vn_{t*}, FPCR)	M
FRINTX	vd_{HSD}, vn_d	Vd_{t*} = Round2IntEx(Vn_{t*},FPCR)	M
sCVTF	vd_{HSD}, vn_d	Vd_{t*} = Int2Float(Vn_{t*}, s)	M,R_n
sCVTF	vd_{HSD}, vn_d, #i	Vd_{t*} = Fixed2Float(Vn_{t*}, i, s)	M,R_n

Floating Point Comparison Instructions			
FCCMP{E}	qn_{HSD},qm_d,#i,cc	NZCV= cc ?(Vn_{t0}<=>Vm_{t0}) : i	
FCMP{E}	qn_{HSD}, qm_d	NZCV = (Vn_{t0} <=> Vm_{t0})	
FCMP{E}	qn_{HSD}, #0.0	NZCV = (Vn_{t0} <=> 0.0)	
FCMcf	vd_{HSD}, vn_d, vm_d	Vd_{t*} = (Vn_{t*} cf Vm_{t*}) ? 1_z : 0_z	M
FCMcz	vd_{HSD}, vn_d, #0.0	Vd_{t*} = (Vn_{t*} cz 0.0) ? 1_z : 0_z	M
FMAX{NM}	vd_{HSD}, vn_d, vm_d	$Vd_{t*} = Vn_{t*} \wedge Vm_{t*}$	M
FMAX{NM}P	vd_{HSD}, vn_d, vm_d	$Vd_{t*}=(Vm:Vn)_{t2*}\wedge(Vm:Vn)_{t2*+1}$	M
FMAX{NM}V	qd_{HS}, vn_{HS}	$Vd_{t0}= Vn_{t0} \wedge Vn_{t1} \wedge ... \wedge Vn_{te}$	
FMIN{NM}	vd_{HSD}, vn_d, vm_d	$Vd_{t*} = Vn_{t*} \vee Vm_{t*}$	M
FMIN{NM}P	vd_{HSD}, vn_d, vm_d	$Vd_{t*}=(Vm:Vn)_{t2*}\vee(Vm:Vn)_{t2*+1}$	M
FMIN{NM}V	qd_{HS}, vn_{HS}	$Vd_{t0}= Vn_{t0} \vee Vn_{t1} \vee ... \vee Vn_{te}$	

Floating Point Misc Instructions			
BFDOT	vd_S,vn_H,vm_n	$Vd_{S*}= Vn_{H2*}\times Vm_{H2*} + Vn_{H2*+1}\times Vm_{H2*+1}$	6,I
BFMLALB	Vd.4S,Vn.8H,Vm.8H	$Vd_{S*} += Vn_{H2*} \times Vm_{H2*}$	6,I
BFMLALT	Vd.4S,Vn.8H,Vm.8H	$Vd_{S*} += Vn_{H2*+1} \times Vm_{H2*+1}$	6,I
BFMMLA	Vd.4S,Vn.8H,Vm.8H	$Vd_{S[2;2]} += Vn_{H[2;4]} \times Vm_{H[4;2]}$	6
FCADD	vd_{HSD},vn_d,vm_d,#r	$Vd_{t*} = Vn_{t*} + CpRot(Vm_{t*\oplus 1}$,r)	3
FCMLA	vd_{HSD},vn_d,vm_d,#r	$Vd_{t[2]*}$+=CpMul($Vn_{t[2]*}$,$Vm_{t[2]*}$,r)	3,I
FRECPE	vd_{HSD}, vn_d	Vd_{t*} = RecipEstimate(Vn_{t*})	M
FRECPS	vd_{HSD}, vn_d, vm_d	Vd_{t*} = RecipStep(Vn_{t*},Vm_{t*})	M
FRECPX	qd_{HSD}, qn_d	Vd_{t0} = RecipExponent(Vn_{t0})	
FRSQRTE	vd_{HSD}, vn_d	Vd_{t*} = RecipSqrtEstimate(Vn_{t*})	M
FRSQRTS	vd_{HSD}, vn_d, vm_d	Vd_{t*}=RecipSqrtStep(Vn_{t*},Vm_{t*})	M

SIMD Arithmetic Instructions

ABS	vd_{BHSD}, vn_d	$Vd_{t*} = \lvert Vn_{t*}\rvert$	M_D
ADD	vd_{BHSD}, vn_d, vm_d	$Vd_{t*} = Vn_{t*} + Vm_{t*}$	M_D
ADDP	vd_{BHSD}, vn_d, vm_d	$Vd_{t*}=(Vm{:}Vn)_{t2*}+(Vm{:}Vn)_{t2*+1}$	M_D
ADDV	qd_{BHS}, vn_d	$Vd_{t0} = Vn_{t0} + Vn_{t1} + ... + Vn_{te}$	
NEG	vd_{BHSD}, vn_d	$Vd_{t*} = -Vn_{t*}$	M_D
{R}ADDHN{2}	vd_{BHS},vn_{BHS},vm_n	$Vd_{t*} = (Vn_{t'*} + Vm_{t'*}) \gg^{\{R\}} z$	
{R}SUBHN{2}	vd_{BHS},vn_{BHS},vm_n	$Vd_{t*} = (Vn_{t'*} - Vm_{t'*}) \gg^{\{R\}} z$	
sABA	vd_{BHS}, vn_d, vm_d	$Vd_{t*} \mathrel{+}= \lvert Vn^s_{t*} - Vm^s_{t*}\rvert$	
sABAL{2}	vd_{HSD},vn_{BHS},vm_n	$Vd_{t*} \mathrel{+}= \lvert Vn^s_{t'*} - Vm^s_{t'*}\rvert$	
sABD	vd_{BHS}, vn_d, vm_d	$Vd_{t*} = \lvert Vn^s_{t*} - Vm^s_{t*}\rvert$	
sABDL{2}	vd_{HSD},vn_{BHS},vm_n	$Vd_{t*} = \lvert Vn^s_{t'*} - Vm^s_{t'*}\rvert$	
sADALP	vd_{HSD}, vn_{BHS}	$Vd_{t*} \mathrel{+}= Vn^s_{t2*} + Vn^s_{t2*+1}$	
sADDL{2}	vd_{HSD},vn_{BHS},vm_n	$Vd_{t*} = Vn^s_{t'*} + Vm^s_{t'*}$	
sADDLP	vd_{HSD}, vn_{BHS}	$Vd_{t*} = Vn^s_{t'2*} + Vn^s_{t'2*+1}$	
sADDLV	qd_{HSD}, vn_{BHS}	$Vd_{t0} = Vn^s_{t'0} + Vn^s_{t'1} + ... + Vn^s_{t'e}$	
sADDW{2}	vd_{HSD},vn_d,vm_{BHS}	$Vd_{t*} = Vn_{t*} + Vm^s_{t'*}$	
sHSUB	vd_{BHS}, vn_d, vm_d	$Vd_{t*} = (Vn_{t*} + Vm_{t*}) \gg^s 1$	
SQABS	vd_{BHSD}, vn_d	$Vd_{t*} = \lfloor \lvert Vn_{t*}\rvert \rceil^{Sz}$	M
sQADD	vd_{BHSD}, vn_d, vm_d	$Vd_{t*} = \lfloor Vn_{t*} + Vm_{t*} \rceil^{sz}$	M
SQNEG	vd_{BHSD}, vn_d	$Vd_{t*} = \lfloor -Vn \rceil^{Sz}$	M
sQSUB	vd_{BHSD}, vn_d, vm_d	$Vd_{t*} = \lfloor Vn^s_{t*} - Vm^s_{t*} \rceil^{sz}$	M
sQXTN{2}	vd_{BHS}, vn_{HSD}	$Vd_{t*} = \lfloor Vn_{t'*} \rceil^{szd}$	M
SQXTUN{2}	vd_{BHS}, vn_{HSD}	$Vd_{t*} = \lfloor Vn^{\pm}_{t'*} \rceil^{Uzd}$	M
s{R}HADD	vd_{BHS}, vn_d, vm_d	$Vd_{t*} = (Vn_{t*} + Vm_{t*}) \gg^{s\{R\}} 1$	
sSUBL{2}	vd_{HSD},vn_{BHS},vm_n	$Vd_{t*} = Vn^s_{t'*} - Vm^s_{t'*}$	
sSUBW{2}	vd_{HSD},vn_d,vm_{BHS}	$Vd_{t*} = Vn_{t*} - Vm^s_{t'*}$	
SUB	vd_{BHSD}, vn_d, vm_d	$Vd_{t*} = Vn_{t*} - Vm_{t*}$	M_D
SUQADD	vd_{BHSD}, vn_d	$Vd_{t*} = \lfloor Vd^{\pm}_{t*} + Vn^{\emptyset}_{t*} \rceil^{Sz}$	M
USQADD	vd_{BHSD}, vn_d	$Vd_{t*} = \lfloor Vd^{\emptyset}_{t*} + Vn^{\pm}_{t*} \rceil^{Uz}$	M

SIMD Multiplication Instructions

MLa	vd_{BHS}, vn_d, vm_d	$Vd_{t*} \pm= Vn_{t*} \times Vm_{t*}$	I_{HS}
MUL	vd_{BHS}, vn_d, vm_d	$Vd_{t*} = Vn_{t*} \times Vm_{t*}$	I_{HS}
sMLaL{2}	vd_{SD}, vn_{HS}, vm_d	$Vd_{t*} \pm= Vn_{t'*} \times^s Vm_{t'*}$	I
sMULL{2}	vd_{SD}, vn_{HS}, vm_d	$Vd_{t*} = Vn_{t'*} \times^s Vm_{t'*}$	I
SQDMLaL{2}	vd_{SD},vn_{HS},vm_d	$Vd_{t*} \pm= \lfloor 2 \bar{\times} Vn_{t'*} \bar{\times} Vm_{t'*} \rceil^{Sz}$	I,M
SQDMULL{2}	vd_{SD},vn_{HS},vm_d	$Vd_{t*} = \lfloor 2 \bar{\times} Vn_{t'*} \bar{\times} Vm_{t'*} \rceil^{Sz}$	I,M
SQRDMaLH	vd_{HS}, vn_d, vm_d	$Vd_{t*}=\lfloor Vd_{t*} \pm (2 \bar{\times} Vn_{t*} \bar{\times} Vm_{t*} \gg^R z$	1,I,M
SQ{R}DMULH	vd_{HS}, vn_d, vm_d	$Vd_{t*}=\lfloor 2 \bar{\times} Vn_{t*} \bar{\times} Vm_{t*} \gg^{\{R\}} z \rceil^{Sz}$	I,M

Notes for Floating Point and SIMD Instruction Set

I	Third operand may be indexed, e.g. Vm.t[i]
M, M_t	Scalar may be used instead of vector, types may be limited to t
O	Optional feature
R_d, R_n	General register may be used as operand d or n, implies M
1,2,3,4,5,6	Introduced in ARMv8.{1..6}

SIMD Shift Instructions

{R}SHRN{2}	vd_{BHS},vn_{BHS},#i	$Vd_{t*} = Vn_{t'*} \gg^{\{R\}} i$	
SHL	vd_{BHSD}, vn_d, #i	$Vd_{t*} = Vn_{t*} \ll i$	M_D
SHLL{2}	vd_{HSD},vn_{BHS},#i	$Vd_{t*} = Vn_{t'*} \ll i$	
sQ{R}SHL	vd_{BHSD}, vn_d, vm_d	$Vd_{t*}=\lfloor Vn_{t*} \ll\gg^{s\{R\}} Vm_{t*7:0} \rceil^{sz}$	M
sQ{R}SHRN{2}	vd_{BHS},vn_{BHS},#i	$Vd_{t*} = \lfloor Vn_{t'*} \gg^{s\{R\}} i \rceil^{sz}$	M
SQ{R}SHRUN{2}	vd_{BHS},vn_{BHS},#i	$Vd_{t*} = \lfloor Vn_{t'*} \bar{\gg}^{\{R\}} i \rceil^{Uz}$	M
sQSHL	vd_{BHSD}, vn_d, #i	$Vd_{t*} = \lfloor Vn_{t*} \ll^s i \rceil^{sz}$	M
SQSHLU	vd_{BHSD}, vn_d, #i	$Vd_{t*} = \lfloor Vn_{t*} \ll^S i \rceil^{Uz}$	M
s{R}SHL	vd_{BHSD}, vn_d, vm_d	$Vd_{t*} = Vn_{t*} \ll \bar{\gg}^{\{R\}} Vm_{t*7:0}$	M_D
s{R}SHR	vd_{BHSD}, vn_d, #i	$Vd_{t*} = Vn_{t*} \gg^{s\{R\}} i$	M_D
s{R}SRA	vd_{BHSD}, vn_d, #i	$Vd_{t*} \mathrel{+}= Vn_{t*} \gg^{s\{R\}} i$	M_D
sSHLL{2}	vd_{HSD},vn_{BHS},#i	$Vd_{t*} = Vn^s_{t'*} \ll i$	

SIMD Bitwise Instructions

CLS	vd_{BHS}, vn_d	$Vd_{t*} = \text{CountLeadingSigns}(Vn_{t*})$	
CLZ	vd_{BHS}, vn_d	$Vd_{t*} = \text{CountLeadingZeros}(Vn_{t*})$	
CNT	vd_B, vn_d	$Vd_{B*} = \text{CountOneBits}(Vn_{B*})$	
EXT	vd_B,vn_d,vm_d,#i_4	$Vd_{B*} = Vm_{B*[i-1:0]}{:}Vn_{B*[z-1:z-i+1]}$	
RBIT	vd_B, vn_d	$Vd_{B*} = \text{ReverseBits}(Vn_{B*})$	
REV16	vd_B, vn_d	$Vd_{B*} = Vn_{B*\oplus 1}$	
REV32	vd_{BH}, vn_d	$Vd_{t*} = Vn_{t*\oplus(32\div z-1)}$	
REV64	vd_{BHS}, vn_d	$Vd_{t*} = Vn_{t*\oplus(64\div z-1)}$	
SLI	vd_{BHSD}, vn_d, #i	$Vd_{t*[z-1:i]} = Vn_{t*[z-i-1:0]}$	M_D
SRI	vd_{BHSD}, vn_d, #i	$Vd_{t*[z-i-1:0]} = Vn_{t*[z-1:i]}$	M_D
TRN1	vd_{BHSD}, vn_d, vm_d	$Vd_{t*} = \text{iseven}(*)\ ?\ Vn_{t*} : Vm_{t*-1}$	
TRN2	vd_{BHSD}, vn_d, vm_d	$Vd_{t*} = \text{iseven}(*)\ ?\ Vn_{t*+1} : Vm_{t*}$	
UZP1	vd_{BHSD}, vn_d, vm_d	$Vd_{t*} = (Vm{:}Vn)_{t2*}$	
UZP2	vd_{BHSD}, vn_d, vm_d	$Vd_{t*} = (Vm{:}Vn)_{t2*+1}$	
ZIP1	vd_{BHSD}, vn_d, vm_d	$Vd_{t*}= \text{iseven}(*)\ ?\ Vn_{t*\div 2} : Vm_{t*\div 2}$	
ZIP2	vd_{BHSD}, vn_d, vm_d	$Vd_{t*}=\text{iseven}(*)\ ?Vn_{tz+(*\div 2)} :Vm_{tz+(*\div 2)}$	

Keys for Floating Point and SIMD Instructions

vd_t, vn_t, vm_t	SIMD register as vector array of type t
qd_t, qn_t, qm_t	SIMD register as scalar of type t
a	A for addition or S for subtraction
e	Last element in vector, i.e. number of elements minus one
r	Rounding (A, M, N, P, or Z)
s	Operation signess (S or U)
t	Operand type (B, H, S, D, or Q)
t'	Secondary operand type when more than one type is used
z	Vector operation size (8, 16, 32, or 64)
{2}	Optional suffix to use upper half of a register. Otherwise use lower half and clear upper half if destination
cs	SIMD register comparison operator (EQ, GE, GT, HI, or HS)
cz	SIMD zero comparison operator (EQ, GE, GT, LE, or LT)
cf	SIMD float comparison operator (EQ, GE, or GT)

SIMD Move Instructions

DUP	vd_{BHSD}, Vn.t[i]	$Vd_{t*} = Vn_{ti}$
DUP	vd_{BHSD}, rn	$Vd_{t*} = rn_{t0}$
MOV	$Vd.t_{BHSD}$[i], Vn.t[j]	$Vd_{ti} = Vn_{tj}$
MOV	$Vd.t_{BHSD}$[i], rn	$Vd_{ti} = rn_{t0}$
MOV	qd_{BHSD}, Vn.t[i]	$Vd_{t0} = Vn_{ti}$
MOV	rd, $vn.t_{SD}$[i]	$rd = Vn_{ti}$
MOV	vd_B, vn_d	$Vd_{B*} = Vn_{B*}$
MOVI	Dd, #i	$Vd_{D0} = i$
MOVI	Vd.2D, #i	$Vd_{D*} = i$
MOVI	vd_{BHS},#i_8{,LSL #sh}	$Vd_{t*} = i^{\emptyset} \ll sh$
MOVI	vd_S, #i_8, MSL #sh	$Vd_{S*} = i^{\emptyset}{:}1_{sh}$
MVNI	vd_{HS}, #i_8{, LSL #sh}	$Vd_{t*} = \sim(i^{\emptyset} \ll sh)$
MVNI	vd_S, #i_8, MSL #sh	$Vd_{S*} = \sim(i^{\emptyset}{:}1_{sh})$
sMOV	Wd, $Vn.t_{BH}$[i]	$Wd = Vn^s_{ti}$
sXTL{2}	vd_{HSD}, vn_{BHS}	$Vd_{t*} = Vn^s_{t'*}$
TBL	vd_B,{Vn.16B, ...},vm_d	$Vd_{B*}= Vm_{B*}<16 \text{cnt}\ ?(...{:}Vn)_{BV_{B*}} :0$
TBX	vd_B,{Vn.16B, ...},vm_d	$\text{if}(Vm_{B*}<16\text{cnt})\ Vd_{B*} = (...{:}Vn)_{BV_{B*}}$
XTN{2}	vd_{BHS}, vn_{HSD}	$Vd_{t*} = Vn_{t'*}$

SIMD Logical Instructions

AND	vd_B, vn_d, vm_d	$Vd_{B*} = Vn_{B*}\ \&\ Vm_{B*}$
BIC	vd_B, vn_d, vm_d	$Vd_{B*} = Vn_{B*}\ \&\ \sim Vm_{B*}$
BIC	vd_{HS}, #i_8{, LSL #sh}	$Vd_{t*}\ \&= \sim(i \ll sh)$
BIF	vd_B, vn_d, vm_d	$Vd_{B*}=(Vd_{B*}\&\sim Vm_{B*})\lvert(Vn_{B*}\&Vm_{B*})$
BIT	vd_B, vn_d, vm_d	$Vd_{B*}=(Vd_{B*}\&Vm_{B*})\lvert(Vn_{B*}\&\sim Vm_{B*})$
BSL	vd_B, vn_d, vm_d	$Vd_{B*}=(Vn_{B*}\&Vd_{B*})\lvert(Vm_{B*}\&\sim Vd_{B*})$
EOR	vd_B, vn_d, vm_d	$Vd_{B*} = Vn_{B*} \oplus Vm_{B*}$
MVN	vd_B, vn_d	$Vd_{B*} = \sim Vn_{B*}$
ORN	vd_B, vn_d, vm_d	$Vd_{B*} = Vn_{B*} \mid \sim Vm_{B*}$
ORR	vd_B, vn_d, vm_d	$Vd_{B*} = Vn_{B*} \mid Vm_{B*}$
ORR	vd_{HS}, #i_8{, LSL #sh}	$Vd_{t*} \mid= i^{\emptyset} \ll sh$

SIMD Comparison Instructions

CMcs	vd_{BHSD}, vn_d, vm_d	$Vd_{t*} = (Vn_{t*}\ \text{cs}\ Vm_{t*})\ ?\ 1_z : 0_z$	M_D
CMcz	vd_{BHSD}, vn_d, #0	$Vd_{t*} = (Vn_{t*}\ \text{cz}\ 0)\ ?\ 1_z : 0_z$	M_D
CMTST	vd_{BHSD}, vn_d, vm_d	$Vd_{t*} = ((Vn_{t*}\ \&\ Vm_{t*}) \neq 0)\ ?\ 1_z : 0_z$	M_D
sMAX	vd_{BHS}, vn_d, vm_d	$Vd_{t*} = Vn_{t*} \wedge^s Vm_{t*}$	
sMAXP	vd_{BHS}, vn_d, vm_d	$Vd_{t*} = (Vm{:}Vn)_{t2*} \wedge^s (Vm{:}Vn)_{t2*+1}$	
sMAXV	qd_{BHS}, vn_d	$Vd_{t0} = Vn_{t0} \wedge^s Vn_{t1} \wedge^s ... \wedge^s Vn_{te}$	
sMIN	vd_{BHS}, vn_d, vm_d	$Vd_{t*} = Vn_{t*} \vee^s Vm_{t*}$	
sMINP	vd_{BHS}, vn_d, vm_d	$Vd_{t*} = (Vm{:}Vn)_{t2*} \vee^s (Vm{:}Vn)_{t2*+1}$	
sMINV	qd_{BHS}, vn_d	$Vd_{t0} = Vn_{t0} \vee^s Vn_{t1} \vee^s ... \vee^s Vn_{te}$	

ARMv8-A System

Control Registers		
GMID_EL1	Multiple Tag Transfer ID	5
SCTLR_EL{1..3}	System Control	
ACTLR_EL{1..3}	Auxiliary Control	
CPACR_EL1	Architectural Feature Access Control	
RGSR_EL1	Random Allocation Tag Seed	O,5
GCR_EL1	Tag Control	O,5
H{A}CR_EL2	Hypervisor {Auxiliary} Configuration	
SCR_EL3	Secure Configuration	
CPTR_EL{2,3}	Architectural Feature Trap	
HSTR_EL2	Hypervisor System Trap	
HFG{R,W}TR_EL2	Hyp Fine-Grained {Rd,Wr} Trap	6
HFGITR_EL2	Hyp Fine-Grained Instruction Trap	6
AP{I,D}kKey_{Lo,Hi}_EL1	Pointer Auth Key k for {Instr,Data}	3
VNCR_EL2	Virtual Nested Control	O,4
APGAKey_{Lo,Hi}_EL1	Pointer Auth Key A for Code	3
RNDR{RS}	{Reseeded} Random Number	O,RO,5
HDFGRT{R,W}_EL2	Hyp Dbg Fine-Grain {Rd,Wr} Trap	6
HAFGRTR_EL2	Hyp Act Mon Fine-Grain Read Trap	6
LOR{S,E}A_EL1	LORegion {Start,End} Address	1
LOR{C,N,ID}_EL1	LORegion {Control,Number,ID}	1

Translation Registers	
TTBR0_EL{1..3}	Translation Table Base 0 (4K/16K/64K align)
TTBR1_EL{1..2}	Translation Table Base 1 (4K/16K/64K align)
TCR_EL{1..3}	Translation Control
V{S}TTBR_EL2	Virt {Secure} Translation Table Base
V{S}TCR_EL2	Virt {Secure} Translation Control
{A}MAIR_EL{1..3}	{Auxiliary} Memory Attr Indirection

Exception Registers		
AFSR{0,1}_EL{1..3}	Auxiliary Fault Status {0,1}	
ESR_EL{1..3}	Exception Syndrome	
TFSR_EL{1..3}	Tag Fault Status	O,5
TFSRE0_EL1	EL0 Tag Fault Status	O,5
FAR_EL{1..3}	Fault Address	
HPFAR_EL2	Hypervisor IPA Fault Address	
PAR_EL1	Physical Address	
VBAR_EL{1..3}	Vector Base Address (2k aligned)	
RVBAR_EL{1..3}	Reset Vector Base Address	RO
RMR_EL{1..3}	Reset Management	
ISR_EL1	Interrupt Status	RO

ID Registers		
MIDR_EL1	Main ID	RO
MPIDR_EL1	Multiprocessor Affinity	RO
REVIDR_EL1	Revision ID	RO
CCSIDR{,2}_EL1	Current Cache Size ID {1,2}	RO
CLIDR_EL1	Cache Level ID	RO
AIDR_EL1	Auxiliary ID	RO
CSSELR_EL1	Cache Size Selection	
CTR_EL0	Cache Type	RO
DCZID_EL0	Data Cache Zero ID	RO
VPIDR_EL2	Virtualization Processor ID	
VMPIDR_EL2	Virtualization Multiprocessor ID	
ID_AA64PFR{0,1}_EL1	AArch64 Processor Feature {0,1}	RO
ID_AA64DFR{0,1}_EL1	AArch64 Debug Feature {0,1}	RO
ID_AA64AFR{0,1}_EL1	AArch64 Auxiliary Feature {0,1}	RO
ID_AA64ISAR{0,1}_EL1	AArch64 Instruction Set Attribute {0,1}	RO
ID_AA64MMFR{0,2}_EL1	AArch64 Memory Model Feature {0,1}	RO
CONTEXTIDR_EL{1,2}	Context ID	
TPIDR_EL{0..3}	Software Thread ID	
TPIDRRO_EL0	EL0 Read-only Software Thread ID	
SCXTNUM_EL{0..3}	Software context number	

Generic Timer Registers		
CNTFRQ_EL0	Ct Frequency (in Hz)	
CNT{P,V}CT_EL0	Ct {Physical,Virtual} Count	RO
CNTVOFF_EL2	Ct Virtual Offset	
CNT{P,V}CTSS_EL0	Ct Self-Sync {Phys,Virt} Count	6
CNTPOFF_EL2	Ct Physical Offset	6
CNTKCTL_EL1	Ct Kernel Control	
CNTHCTL_EL2	Ct Hypervisor Control	
CNT{P,V}_{TVAL,CTL,CVAL}_EL0	Ct {Physical,Virtual} Timer	
CNTHP_{TVAL,CTL,CVAL}_EL2	Ct Hypervisor Physical Timer	
CNTPS_{TVAL,CTL,CVAL}_EL1	Ct Physical Secure Timer	
CNTHV_{TVAL,CTL,CVAL}_EL2	Ct Virtual Timer	1
CNTHVS_{TVAL,CTL,CVAL}_EL2	Ct Secure Virtual Timer	4
CNTHPS_{TVAL,CTL,CVAL}_EL2	Ct Hyp Secure Physical Timer	4

Statistical Profiling Registers (ARMv8.3 Optional)		
PMSCR_EL{1,2}	Statistical Profiling Control	
PMSI{C,R}_EL1	Sampling Interval {Counter,Reload}	
PMSFCR_EL1	Sampling Filter Control	
PMS{EV,LAT}FR_EL1	Sampling Event/Latency Filter	
PMSIDR_EL1	Sampling Profiling ID	RO
PMBLIMITR_EL1	Profiling Buffer Limit Address	
PMBPTR_EL1	Profiling Buffer Write Pointer	
PMBSR_EL1	Profiling Buffer Status/syndrome	
PMBIDR_EL1	Profiling Buffer ID	RO

Performance Monitors Registers		
PMCR_EL0	PM Control	
PMCNTEN{SET,CLR}_EL0	PM Count Enable {Set,Clear}	
PMOVSCLR_EL0	PM Overflow Flag Status Clear	
PMSWINC_EL0	PM Software Increment	WO
PMSELR_EL0	PM Event Counter Selection	
PMCEID{0,1}_EL0	PM Common Event ID {0,1}	RO
PMCCNTR_EL0	PM Cycle Count Register	
PMXEVTYPER_EL0	PM Selected Event Type	
PMXEVCNTR_EL0	PM Selected Event Count	
PMUSERENR_EL0	PM User Enable	
PMINTEN{SET,CLR}_EL1	PM Interrupt Enable {Set,Clear}	
PMOVSSET_EL0	PM Overflow Flag Status Set	
PMMIR_EL1	PM Machine Identification	O,RO,4
PMEVCNTR{0..30}_EL0	PM Event Count {0..30}	
PMEVTYPER{0..30}_EL0	PM Event Type {0..30}	
PMCCFILTR_EL0	PM Cycle Count Filter	

Reliability, Availability, and Serviceability Registers (Optional)		
VSESR_EL2	Virtual SError Exception Syndrome	
ERRIDR_EL1	Error Record ID	RO
ERRSELR_EL1	Error Record Select	
ERXFR_EL1	Selected Error Record Feature	RO
ERXCTLR_EL1	Selected Error Record Control	
ERXSTATUS_EL1	Selected Error Record Primary Status	
ERXADDR_EL1	Selected Error Record Address	
ERXPFGF_EL1	Selected Pseudo-fault Generation Feature	RO
ERXPFGCTL_EL1	Selected Pseudo-fault Generation Control	
ERXPFGCDN_EL1	Selected Pseudo-fault Generation Countdown	
ERXMISC{0..3}_EL1	Selected Error Record Miscellaneous {0..3}	
DISR_EL1	Deferred Interrupt Status	
VDISR_EL2	Virtual Deferred Interrupt Status	

Activity Monitors Registers (ARMv8.4 Optional)		
AMCR_EL0	AM Control	
AMCFGR_EL0	AM Configuration	
AMCGCR_EL0	AM Counter Group Configuration	
AMUSERENR_EL0	AM User Enable	
AMCNTENCLR{0,1}_EL0	AM Count Enable Clear {0,1}	
AMCNTENSET{0,1}_EL0	AM Count Enable Set {0,1}	
AMCG1IDR_EL0	AM Counter Group 1 Identification	RO
AMEVCNTR{0,1}{0..15}_EL0	AM Event Counter {0,1} {0..15}	
AMEVTYPER{0,1}{0..15}_EL0	AM Event Type {0,1} {0..15}	
AMEVCNTVOFF{0,1}{0..15}_EL2	AM Ev Cnt {0,1} {0..15} Virt Off	

Translation Control Register with EL0 (TCR_EL{1,2})			
T{1,0}SZ	0x00000000_003F003F	Region size	
A1	0x00000000_00400000	ASID definition (Table 0,Table 1)	
EPD{1,0}	0x00000000_00800080	Disable trans table walks	
IRGN{1,0}	0x00000000_03000300	Inner cache (no,wb+wa,wt,wb)	
ORGN{1,0}	0x00000000_0C000C00	Outer cache (no,wb+wa,wt,wb)	
SH{1,0}	0x00000000_30003000	Shareability (no,-,out,in)	
TG{1,0}	0x00000000_C000C000	Granule size (4K,64K,16K,-)	
IPS	0x00000007_00000000	Phys spc (32,36,40,42,44,48,52,-)	
AS	0x00000010_00000000	ASID size (8,16)	
TBI{1,0}	0x00000040_00000000	Top instruction addr byte ignored	
HA	0x00000080_00000000	HW managed access flag	1
HD	0x00000100_00000000	HW managed dirty state	1
HPD{1,0}	0x00000400_00000000	No hierarchical perm	1
HWU{1,0}b	0x007F8000_00000000	Bit {59..62} used by HW	2
TBID{1,0}	0x00180000_00000000	TBI{1,0} affects only data	3
NFD{1,0}	0x00600000_00000000	No non-fault table walks	O
E0PD{1,0}	0x01800000_00000000	Fault on unpriv access	5
TCMA0	0x02000000_00000000	Tag 0x00 unchecked	5
TCMA1	0x04000000_00000000	Tag 0x1f unchecked	5

Translation Control Register without EL0 (TCR_EL{2,3})			
T0SZ	0x0000003F	Memory region size	
IRGN0	0x00000300	Inner cache (non,wb+wa,wt,wb)	
ORGN0	0x00000F00	Outer cache (non,wb+wa,wt,wb)	
SH0	0x00003000	Shareability (non,-,outer,inner)	
TG0	0x0000C000	Granule size (4K,64K,16K,-)	
PS	0x00070000	Physical space (32,36,40,44,84,52,-)	
TBI	0x00100000	Top byte ignored	
HA	0x00200000	HW managed access flag	O,1
HD	0x00400000	HW managed dirty state	O,1
HPD	0x01000000	No hierarchial permissions	1
HWU{59..62}	0x1E000000	Bit {59..62} used by HW	O,2
TBID	0x20000000	TBI affects only data	3
TCMA	0x40000000	Tag 0x00 unchecked	5

Virtual Memory Block table format			
V	0x00000000_00000001	Descriptor valid (always 1)	
D	0x00000000_00000002	Descriptor type (always 1)	
OA48	0x00000000_0000F000	Output address bits 12-15 or 48-51	
OA	0x0000FFFF_FFFF0000	Next-level table address, bits 16-47	
PXN	0x08000000_00000000	Privileged execute never	E1
{U}XN	0x10000000_00000000	{EL0} Execute never	
AP0	0x20000000_00000000	Access not permitted at EL0	E1
AP1	0x40000000_00000000	Write access not permitted	
NS	0x80000000_00000000	Non-secure access	

Secure Configuration Register (SCR)			
NS	0x0_00000001	System state is non-secure unless in EL3	
IRQ	0x0_00000002	IRQs taken to EL3	
FIQ	0x0_00000004	FIQs taken to EL3	
EA	0x0_00000008	External aborts and SError taken to EL3	
SMD	0x0_00000080	Secure monitor call disable	
HCE	0x0_00000100	Hyp Call enable	
SIF	0x0_00000200	Secure instruction fetch	
RW	0x0_00000400	Lower level is AArch64	
ST	0x0_00000800	Trap secure EL1 to CNTPS registers to EL3	
TWI	0x0_00001000	Trap EL{0..2} WFI instruction to EL3	
TWE	0x0_00002000	Trap EL{0..2} WFE instruction to EL3	
TLOR	0x0_00004000	Trap LOR registers	1
TERR	0x0_00008000	Trap error record accesses	2
APK	0x0_00010000	Don't trap Pointer Auth keys	3
API	0x0_00020000	Don't trap Pointer Auth instructions	3
EEL2	0x0_00040000	Secure EL2	4
EASE	0x0_00080000	External Aborts to SError	4
NMEA	0x0_00100000	Non-maskable External Aborts	4
FIEN	0x0_00200000	Fault Injection	4
EnSCXT	0x0_02000000	STCXTNUM_ELx access	
ATA	0x0_04000000	Allocation Tag Access	5
FGTEn	0x0_08000000	Fine-Grained Traps	6
ECVEn	0x0_10000000	CNTPOFF_EL2 access	6
TWEDEn	0x0_20000000	TWE Delay	6
TWEDEL	0x2_C0000000	TWE Delay Length	6
AMVOFFEN	0x8_00000000	Activity Monitor Virtual Offsets	6

Virtual Memory Block/Page descriptor format			
V	0x00000000_00000001	Descriptor valid (always 1)	
D	0x00000000_00000002	Descriptor type (1 if level 3)	
Attr	0x00000000_0000001C	Attribute index	
NS	0x00000000_00000020	Non-secure	
AP1	0x00000000_00000040	Access from EL0	E1
AP2	0x00000000_00000080	Read-only	
SH	0x00000000_00000300	Shareability (non,-,outer,inner)	
AF	0x00000000_00000400	Access flag	
nG	0x00000000_00000800	Not global	
OA48	0x00000000_0000F000	Output address bits 12-15 or 48-51	
nT	0x00000000_00010000	Block translation entry (level 0-2)	
OA	0x0000FFFF_FFFF0000	Output address, bits 16-47	
GP	0x00040000_00000000	Guarded page	
DBM	0x00080000_00000000	Dirty bit modifier	
C	0x00100000_00000000	Contiguous	
PXN	0x00200000_00000000	Privileged execute never	E1
{U}XN	0x00400000_00000000	{EL0} Execute never	
PBHA	0x78000000_00000000	Page-based hardware attributes	O,2

System Control Register (SCTLR) - Low 32 Bits			
M	0x00000001	MMU enabled	
A	0x00000002	Alignment check enabled	
C	0x00000004	Data and unified caches enabled	
SA	0x00000008	Enable SP alignment check	
SA0	0x00000010	Enable EL0 SP alignment check	E1
nAA	0x00000040	No alignment fault on LDR*R*/STR*R*	4
UMA	0x00000200	Trap EL0 access of DAIF	E1
EnRCTX	0x00000400	EL0 access CFP,DVP,CPP	E1,O
EOS	0x00000800	Sync context on exception return	O,5
I	0x00001000	Instruction cache enabled	
DZE	0x00004000	Trap EL0 DC instruction	E1
UCT	0x00008000	Trap EL0 access of CTR_EL0	E1
nTWI	0x00010000	Trap EL0 WFI instruction	E1
nTWE	0x00040000	Trap EL0 WFE instruction	E1
WXN	0x00080000	Write permission implies XN	
TSCXT	0x00100000	Disable SCXTNUM_EL0	E1
IESB	0x00200000	Implicit error synchronization barrier	O,2
EIS	0x00400000	Sync context on exception entry	O,5
SPAN	0x00800000	Set privileged access never	E1,1
E0E	0x01000000	Data at EL0 is big-endian	E1
EE	0x02000000	Data at EL1 is big-endian	
UCI	0x04000000	Trap EL0 cache instructions	E1
EnD{B,A}	0x08002000	Data pointer auth key {B,A}	3
nTLSMD	0x10000000	No trap load/store multiple to device	E1,2
LSMAOE	0x20000000	Load/store multi atomicity and order	E1,2
EnI{B,A}	0xC0000000	Instruction pointer auth key {B,A}	3

System Control Register (SCTLR) - High 32 Bits			
BT0	0x00000008	EL0 PAC branch type compatility	E1,5
BT{1}	0x00000010	PAC branch type compatility	5
ITFSB	0x00000020	Synchronized tag check faults	5
TCF0	0x000000C0	EL0 tag check fault (no,sync,async,-)	E1,5
TCF	0x00000300	Tag check fault (no,sync,async,-)	5
ATA0	0x00000400	EL0 allocation tag access	E1,5
ATA	0x00000800	Allocation tag access	5
DSSBS	0x00001000	Default SSBS value on exception	O
TWEDEn	0x00002000	TWE delay enable	E1,6
TWEDEL	0x0003C000	TWE delay	E1,6

Notes for System Registers	
RO	Read only
E1	Only present in EL1 register
O	Optional feature
1,2,3,4,5,6	Introduced in ARMv8.{1..6}